Code Review

Joern Ploennigs · AI4SC



Code, der plausibel aussieht, ist noch nicht automatisch korrekt.

Warum KI-Code reviewen?

KI-Werkzeuge wie Claude Code oder GitHub Copilot generieren Code schnell — aber nicht immer korrekt. Ein Code Review ist bei KI-generiertem Code genauso notwendig wie bei menschlichem Code, vielleicht sogar wichtiger: KI zeigt keine Unsicherheit.

KI ist zuversichtlich — nicht immer korrekt

KI ist gut in

- Code-Struktur und Syntax
- Standardmuster implementieren
- Erste Lösungen schnell liefern

KI macht Fehler bei

- Anforderungen vollständig umsetzen
- Randfälle berücksichtigen
- Nicht existierende Funktionen
- Typ-Annahmen validieren

Ein fehlerhafter KI-Code sieht syntaktisch genauso aus wie korrekter Code. KI zeigt keinerlei Unsicherheitssignale — deshalb ist menschliche Prüfung unverzichtbar.

Code Review: Drei Dimensionen

Review-Prozess

1. Anforderungsprüfung

Macht der Code, was verlangt wurde?

2. Designprüfung
Entspricht er der Architektur?

3. Implementierungsprüfung
Ist er korrekt, sicher, wartbar?

Die drei Dimensionen decken unterschiedliche Fehlerquellen ab. Tests automatisieren Teile der Anforderungsprüfung — aber nicht alle. Code Review ergänzt das, was Tests nicht erfassen können: Design-Konflikte und KI-spezifische Halluzinationen.

Anforderungsprüfung

Anforderungsprüfung vergleicht den generierten Code gegen die funktionalen Anforderungen aus der Spezifikation. Jede Anforderung wird einzeln geprüft — keine gilt als implizit erfüllt.

Anforderungen systematisch prüfen

Für jede funktionale Anforderung drei Fragen:

Implementiert?
Gibt es Code, der diese Anforderung umsetzt?

Getestet?
Existiert ein Unit-Test dafür?

Bestanden?
Laufen alle Tests grün durch?

→ `04a_Anforderungen` (S04): Anforderungen als Review-Standard

Die Anforderungen aus der Spezifikation sind der Prüfmaßstab. KI-Code implementiert manchmal nur einen Teil der Anforderungen, lässt andere stillschweigend weg oder interpretiert sie anders als gemeint.

Anforderungs-Tracing: Beispiel

```
# Anforderung: Berechne Neigung in Prozent (Höhe / Länge × 100)

def berechne_neigung(hoehe, laenge):
    return hoehe / laenge   # Fehler: × 100 fehlt — Einheit falsch
```

Was prüfen?

Review-Fragen:

- ☐ Formel korrekt implementiert?
- ☐ Rückgabewert vollständig?
- ☐ Randfälle berücksichtigt?

Test schreiben:

```
# Erwartet: 10.0 (Prozent)
assert berechne_neigung(1, 10) == 10.0
# Ergebnis: 0.1 — Test schlägt fehl
```

Dieses Beispiel zeigt, wie KI-Code syntaktisch korrekt und logisch plausibel wirkt — aber eine Anforderung nur halb erfüllt. Der Test macht den Fehler sichtbar, bevor der Code in Betrieb geht.

Designprüfung

Designprüfung vergleicht den generierten Code gegen das UML-Design und die geplante Modulstruktur. KI kennt das Design nicht — sie erfindet Strukturen, die zwar funktionieren, aber nicht in die geplante Architektur passen.

Design-Match prüfen

Modulstruktur

- Passt der Code ins richtige Modul?
- Werden bestehende Module genutzt?
- Entstehen zirkuläre Importe?

Klasseninterfaces

- Stimmen Klassennamen mit dem UML überein?
- Stimmen Methodensignaturen?
- Werden vorhandene Klassen erweitert?

UML-Design:

```
class Messung:
    + wert: float
    + einheit: str
    + validiere() → bool
```

KI-generierter Code:

```
class DataPoint:       # falscher Name
    def __init__(self, v):
        self.val = v   # falsches Attribut
    # validiere() fehlt
```

→ `03c_Softwareentwurf` (S03): Design als zweiter Review-Standard

Solche Konflikte entstehen, weil KI das Design nicht kennt. Lösung: das UML-Design als Teil des Prompts mitgeben oder den generierten Code gegen das Design anpassen.

Implementierungsprüfung

Implementierungsprüfung scannt den Code auf häufige Fehlerquellen: unsichere Operationen, fehlende Typ-Annotationen, mutable Defaults, hartcodierte Werte und fehlende return-Anweisungen.

Typische Implementierungsprobleme

Division ohne Null-Prüfung

```
def neigung(h, l):
    return h / l  # Fehler: Division durch Null!

# Fix:
def neigung(h, l):
    if l == 0:
        raise ValueError("Länge = 0")
    return h / l
```

Mutable Default-Argument

```
def sammle(w, liste=[]):  # Fehler!
    liste.append(w)
    return liste

# Fix:
def sammle(w, liste=None):
    if liste is None:
        liste = []
    liste.append(w)
    return liste
```

Fehlende Typ-Annotationen

```
def verarbeite(daten):  # Typ unklar
    return daten * 2

# Fix:
def verarbeite(daten: list[float]) -> list[float]:
    return [x * 2 for x in daten]
```

Hartcodierte Werte

```
if temp > 100:  # Warum 100?
    alarm()

# Fix:
MAX_TEMP_CELSIUS = 100
```

```
if temp > MAX_TEMP_CELSIUS:
    alarm()
```

Diese vier Muster sind besonders häufig in KI-generiertem Code. KI-Modelle reproduzieren Muster aus Trainingsdaten — inklusive veralteter oder schlechter Praktiken.

Implementierungs-Scan: Kernchecks

Was bei jedem Code-Block zu prüfen ist:

- ☐ Division: Nullprüfung vorhanden?
- ☐ Standardargumente: kein `list=[]` oder `dict={}`?
- ☐ Typ-Annotationen: alle Parameter annotiert?

Implementierungs-Scan: Ergänzungen

Danach die zweite Prüfrunde:

- ☐ `return`-Anweisung: in jeder Funktion vorhanden?
- ☐ Hartcodierte Werte: durch Konstanten ersetzt?
- ☐ Fehlerbehandlung: sinnvolle Exceptions definiert?

Dieser Scan ist deterministisch — er kann nach jedem KI-generierten Code-Block systematisch durchgeführt werden, ohne Kreativität oder Ermessen zu erfordern.

KI-Halluzinationen

KI-Modelle erfinden manchmal Funktionen, Module und API-Methoden, die nicht existieren. Diese Halluzinationen sind im Code nicht sichtbar — sie treten erst beim Ausführen als `AttributeError` oder `ImportError` auf.

KI-Halluzinationsmuster

Nicht existierende Methoden

```
import pandas as pd

df = pd.read_csv("daten.csv")
df.interpolate_missing()  # existiert nicht
result = df.to_geojson()  # existiert nicht
```

Nicht existierende Module

```
import geopandas_lite      # existiert nicht
import pandas.geospatial   # existiert nicht
```

Falsche API-Nutzung

```
# sklearn falsch verwendet:
model.fit_transform(X_test)     # nur für Training
model.score(X, y, metric="f1")  # kein metric-Parameter
```

Unnötige Abstraktionen

```
class SingletonLoader:   # für Einmalnutzung unnötig
    _instance = None
    def __new__(cls): ...
```

> **i** Erkennungsstrategie
>
> Jeden `import` und jeden Methodenaufruf gegen die Dokumentation prüfen. Code ausführen — nicht nur lesen.

Gegenstrategie: Jeden Import und jeden Methodenaufruf gegen die offizielle Dokumentation prüfen. Code mit kleinen Beispieldaten ausführen, bevor er als korrekt gilt.

KI-Evaluierungscheckliste

Eine strukturierte Checkliste reduziert kognitive Last beim Reviewen von KI-Code. Sie deckt alle drei Review-Dimensionen in sieben Punkten ab.

KI-Evaluierungscheckliste

Anforderungen & Design

- ☐ Anforderungsabdeckung: alle Anforderungen implementiert?
- ☐ Design-Übereinstimmung: Module, Klassen, Signaturen korrekt?

Implementierung

- ☐ Typ-Korrektheit: Annotationen vorhanden und korrekt?
- ☐ Randfälle: Division, leere Listen, None-Werte behandelt?
- ☐ Fehlerbehandlung: sinnvolle Exceptions definiert?

KI-spezifisch

- ☐ Keine halluzierten Aufrufe: alle Funktionen verifiziert?
- ☐ Test-Suite: alle Tests laufen durch?

Agentic Workflow nach der Generierung

Agentic workflow: Nach jeder KI-Generierung: 1. Anforderungen → Checkliste durchgehen 2. Design → UML vergleichen 3. Implementierung → sechs Muster scannen 4. Halluzinationen → Importe und Aufrufe testen 5. Tests → alle grün? 6. Nein → Prompt präzisieren und neu generieren

→ `08a_UnitTest` (S08): Tests als automatisierter Teil des Reviews

Code Review = Verify

Agentic Cycle

Review schließt den Agentic Cycle:

```
Prompt
  ↓
Generate (KI)
  ↓
Understand
  ↓
Verify ← Code Review
  ↓
Iterate (bei Fehler)
```

Review schlägt fehl → iterieren:

- Spezifischen Fehler im Prompt benennen
- Checklisten-Punkt als Prompt-Ergänzung nutzen
- Neu generieren lassen
- Erneut reviewen

> Merksatz
>
> Tests prüfen automatisch. Code Review prüft, was Tests nicht sehen können.

Referenzen

Bibliographie